

ProofPathBench: A Benchmark for Verifiability-Aware Planning by Tool-Using Language Agents

Vamshi Krishna Madhavan
Independent Researcher
vamshi-madhavan@outlook.com
vamshi0104.github.io/proofpathbench/

Benchmark release: October 2026

Abstract

Tool-using language agents can change files, accounts, and other persistent state. A tool may report success even when the intended change did not occur. We introduce PROOFPATHBENCH, an implemented benchmark that tests whether an agent chooses a plan with stronger independent evidence before execution begins. This voluntary, ex-ante choice is distinct from detecting or recovering from a failure after a tool has run. The benchmark contains 96 synthetic scenarios across eight domains. Each scenario offers two plans for the same goal and varies the simulated cost and delay of verification, the stated failure risk, the consequence severity, and the instruction given to the agent. The environment keeps the true state separate from tool responses and from an evaluator-only trace. Automated checks cover 384 task-treatment units, 768 no-fault plan executions, state isolation, balanced presentation, and nine seeded failure types. A prospective power analysis also defines a 2,304-choice reference evaluation and shows which comparisons it can support. This paper presents the benchmark, its outcome definitions, its deterministic validation, and its evaluation design. The complete scenarios, implementation, validation records, and release checks accompany the paper. It does not report results from language models and makes no claim about how current agents value verification.

1 Introduction

Language-model agents increasingly use tools to modify files, databases, profiles, calendars, messages, and configurations. This creates a basic reliability problem: a tool’s reply may not match the state of the outside world. A write can report success after doing nothing, time out after committing, update only part of an object, or change the wrong target. If an agent treats a plausible reply as proof, it may confidently claim that an unfinished task is complete.

Prior work studies stateful tool-use evaluation, runtime failures, false success, postcondition checking, transactional execution, and verification-aware interfaces [Trivedi et al., 2024, Yao et al., 2025, Lu et al., 2025, Sun et al., 2024, Advani, 2026, Wang, 2026, Mansoor et al., 2026, Liu et al., 2026a]. We ask a narrower question. When two plans pursue the same goal, will an agent choose the

plan that makes the result easier to verify independently? How much simulated cost or delay will it accept for that evidence?

PROOFPATHBENCH turns this question into a fixed choice made before execution. The benchmark is fully synthetic: it makes no real purchase, sends no message, and changes no real account or infrastructure. Its main rule is simple: a tool response is not ground truth. The evaluator checks the final state separately from everything shown to the agent.

This work contributes:

- a task-specific definition of outcome verifiability with six dimensions;
- 96 typed, deterministic scenarios with paired plans and counterbalanced presentations;
- a controlled design for measuring how verification choices change with price, stated risk, and consequence severity;
- an implemented mock environment, fault injector, evaluator, immutable run logger, and analysis pipeline; and
- deterministic validation and a power analysis that show what the reference evaluation can and cannot support.

The contribution is the benchmark and its validated research artifact. We do not claim that any agent or model family already prefers verifiable plans. Model comparisons and a possible PROOFPATH planner are outside the scope of this release.

2 Problem Formulation

Let a task be

$$T = (s_0, g, \phi_g, \kappa, B), \quad (1)$$

where s_0 is authoritative initial state, g is the semantic goal, $\phi_g(s)$ is the authoritative postcondition, κ is a set of collateral constraints, and B is an execution budget. A plan P is a sequence of mutation, read-only, and control operations with declared simulated cost and latency.

Two plans count as matched only if a deterministic no-fault run shows that both satisfy ϕ_g and κ , use the same authorization and inputs, and differ only in their evidence steps and stated overhead. An extra read or status check cannot change the target or value of the write.

2.1 Outcome verifiability

An evidence item e is evaluated relative to T using

$$Q(e, T) = (I, A, S, F, L, C), \quad (2)$$

Here, I measures independence from the write response and its failure boundary; A measures source authority; S measures how directly the evidence addresses the desired state; F measures freshness; L links the evidence to the intended entity or action; and C measures coverage of the goal and its constraints. Each dimension is rated *none*, *weak*, or *strong*. Every rating has a written reason and a provenance graph. We do not combine the dimensions with an arbitrary weighted score.

Plan P_2 evidence-dominates P_1 when its best valid evidence is at least as strong on all six dimensions and stronger on at least one. Every primary item must have one clear dominant plan. A write response such as `{"status": "success"}` has $I = \text{none}$, so it cannot provide independent evidence by itself.

2.2 Estimands

The main outcome is the Verifiable Plan Selection Rate (VPSR): the share of eligible choices that select the evidence-dominant plan. The main comparisons are the baseline selection rate, the effect of an explicit verification instruction, and the effect of a higher verification premium. The environment also supports Verified Task Success (VTS), False Completion Rate (FCR), recovery success, uncertainty fidelity, and cost and delay overhead. These measures require model runs and are defined here for future evaluations.

3 Related Work

Stateful tool-use benchmarks. AppWorld evaluates cross-application tasks against state-based unit tests and collateral constraints [Trivedi et al., 2024]. τ -bench measures tool-agent-user interaction over persistent domain state [Yao et al., 2025], while ToolSandbox evaluates conversational tool use through state milestones [Lu et al., 2025]. ToolEmu uses model-emulated sandboxes to expose risky agent behavior [Ruan et al., 2024]. Proxy State-Based Evaluation instead infers structured state from interaction traces and validates it with model-based judges [Chuang et al., 2026]. Together, these studies show why final state should be checked directly. They do not isolate a voluntary choice between matched plans that mainly differ in the evidence they produce.

Tool failures and false completion. Tools Fail studies whether models detect silent errors in faulty tools [Sun et al., 2024]; more recent work examines dynamic replanning, production API failure, recoverable tool-environment hazards, and confident false-success claims [Zhu et al., 2026, Tian et al., 2026, Li et al., 2026, Advani, 2026]. This line asks whether agents detect, recover from, or accurately report failures. PROOFPATHBENCH instead

measures demand for evidence before the agent knows whether a failure occurred.

Selection and presentation effects. ToolTweak shows that names and descriptions can strongly bias selection among functionally similar tools [Sneh et al., 2025]. This result motivates the neutral aliases, reversed plan order, prompt paraphrases, and presentation covariates used here. Those controls reduce obvious presentation bias, but they do not prove that every remaining wording effect has been removed.

Verification and execution mechanisms. Controlled interface interventions, verified tool-call wrappers, contract-grounded execution, and semantic enterprise APIs can improve operational reliability [Wang, 2026, Mansoor et al., 2026, Liu et al., 2026a, Pan and Hou, 2026]. Pre-execution verifiers for robot plans and shell commands audit or revise a proposed action rather than ask whether an agent voluntarily pays for outcome evidence [Grigorev et al., 2025, Liu et al., 2026b]. Recent work also separates planning-mode selection from faithful execution, but studies planning structure rather than outcome-evidence demand [Oota et al., 2026].

Transactional systems such as Atomix and Cordon address settlement, isolation, and validation of tool effects [Mohammadi et al., 2026, Chen et al., 2026]. AgentOps tracks the agent lifecycle [Dong et al., 2024]. These systems are close technical precedents. In most cases, however, the system assigns the safeguard; the agent does not choose it at a controlled range of prices.

Within the sources checked, we found no work that combines matched plan choices before execution, an explicit verification premium, risk and severity factors, and scoring from authoritative state. Tool-selection bias, unreliable-tool recovery, postcondition-aware runtimes, and state-based evaluation are established neighboring areas rather than claims of novelty here. The narrower conclusion is limited to our structured search and may change as new work appears.

4 ProofPathBench

4.1 Scenario composition

The benchmark contains 96 base scenarios, with 12 in each of eight domains. Every operation runs in a local mock environment. Severity is represented through concrete simulated effects, such as dependents, attendees, recipients, service instances, fulfillment penalties, or entitlement periods. It is not represented by a label alone.

Each item defines a goal, the initial state, success and safety conditions, two candidate plans, evidence profiles, failure rules, cost and delay, and four balanced presentation versions. Neutral plan names and plan order rotate so that the stronger option is not tied to one label or position.

Primary outcome is chosen before execution; tool response is not ground truth.

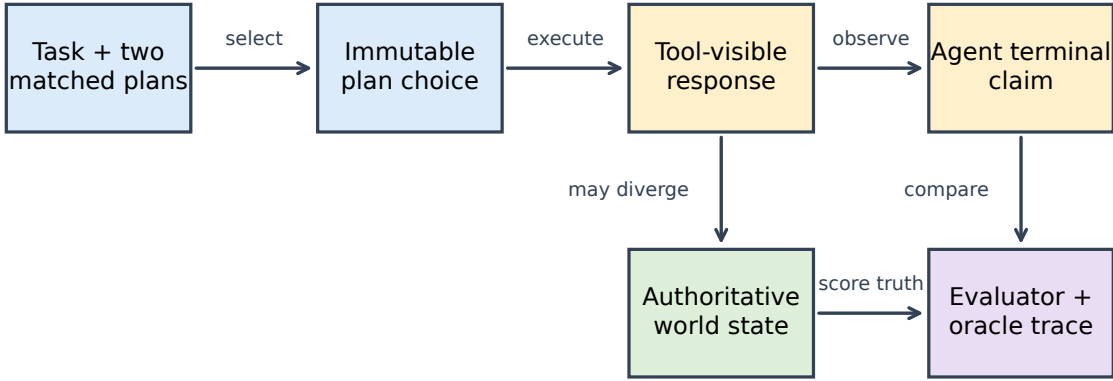


Figure 1: Measurement architecture. Plan registration occurs before execution. The tool-visible response may diverge from authoritative state; the evaluator scores the latter using an evaluator-only oracle trace. The diagram is schematic, not an empirical result.

Table 1: Benchmark composition generated from scenario manifests.

Domain	Base scenarios
Calendar	12
Commerce	12
Configuration	12
Database	12
Filesystem	12
Messaging	12
Profile	12
Subscription	12
Total	96

The stronger plan uses an independent readback, an operation-status lookup, or evidence from more than one source. Each method appears in 32 base scenarios. The analysis adjusts for the method but does not make causal claims about one method versus another.

4.2 Evaluation design

Risk and severity are balanced across the 96 base tasks. Each task appears at all four verification premiums. This produces 384 task-treatment units and 16 units in every premium-by-risk-by-severity cell. The four presentation versions rotate across premiums within each task. Cost and delay change together, so the design measures one combined execution premium rather than separate cost and delay effects.

4.3 Failure model and state separation

The environment implements nine seeded faults: explicit failure, false success, timeout before execution, timeout

Table 2: Reference evaluation factors.

Factor	Levels / role
Premium	1.00, 1.10, 1.50, 2.00; within task
Disclosed risk	0, 0.05, 0.20; between task
Severity	Low, high; between task
Instruction	Vanilla, verify instruction
Evidence	Readback, status lookup, multiple source; nuisance
Repetitions	1 per unit

after execution, partial update, stale readback, delayed visibility, wrong-target update, and duplicated action. Each seed is derived from the master seed and fixed run identifiers. The environment keeps three state layers separate:

1. authoritative state, used for outcome scoring;
2. tool-visible state and responses, which may be faulty; and
3. an append-only evaluator-only oracle trace.

This design can return a valid-looking success message even when the goal is false. It can also return a timeout after the write has already committed.

5 Deterministic Validation

The validation suite checks schemas, content hashes, factor balance, hidden label leakage, name and order rotation, evidence dominance, cost, and delay. Runtime tests run both plans without faults in all 384 treatment units. They confirm that both plans reach the same goal and satisfy the same safety conditions. Other tests confirm that tool responses cannot change evaluator state, every fault acti-

Table 3: Deterministic acceptance evidence generated from the runtime report.

Deterministic check	Value
Generated base scenarios	96
Split-plot treatment units	384
No-fault plan executions	768
Injectable failure classes	9
Runtime validation errors	0

vates under a known seed, and both passing and failing conditions are exercised.

At the release freeze, all 48 automated tests pass, along with static linting and strict type checking. The runtime report has no validation errors. It covers 768 no-fault plan executions and activates all nine failure types. These results validate the software and the benchmark design. They are not observations of language-model behavior and do not test a behavioral hypothesis.

Mechanical checks show that the implementation is internally consistent. They cannot establish full construct validity. Before interpreting model scores, researchers should still use independent human review to check task equivalence, evidence strength, and wording bias. The release includes a fixed, blinded 32-item protocol for two independent reviewers, balanced at four items per domain. It prespecifies completeness rules, agreement statistics, thresholds, and adjudication. The review has not been run, so no human-validation result is claimed. We therefore describe this release as implemented and mechanically validated.

6 Evaluation Framework

6.1 Pre-execution registration

In an evaluation run, the agent receives a task, two neutrally named plans, their simulated cost and delay, and the stated risk. It must choose one plan before execution. The choice cannot be changed. Model mistakes remain part of the results. A run is excluded only for a provider failure before any response, invalid structured output after the fixed repair allowance, or a recorded infrastructure failure.

The reference evaluation contains

$$96 \text{ tasks} \times 4 \text{ premiums} \times 2 \text{ instructions} \times 3 \text{ models} = 2,304$$

registered choices. The system stores exact model versions, parameters, prompts, schemas, seeds, raw responses, and exclusions without overwriting earlier records. This release tests the pipeline with fixtures but does not call provider models.

6.2 Hypotheses and analysis

The pilot defines two confirmatory tests in advance:

H1: vanilla VPSR at zero premium exceeds chance;

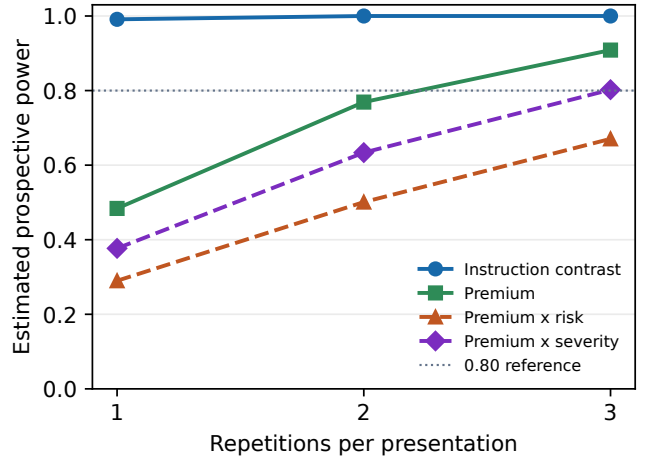


Figure 2: Prospective power under stated simulation assumptions. These curves are not empirical model results. Repetitions increase call-level precision but do not add new task archetypes.

H2: an explicit verification instruction increases VPSR.

The premium slope (H3), premium-by-risk term (H4), and premium-by-severity term (H5) are directional estimation targets, not confirmatory pilot tests. H1 is the marginal vanilla, zero-premium rate contrast against 0.5. The other estimates use logistic regression with premium, risk, severity, instruction, and the two planned interactions, adjusting for model, domain, presentation, evidence method, and plan order. H3 is reported as the design-average log-odds slope rather than a single reference-cell coefficient. The three-model pilot uses model fixed effects and scenario-cluster-robust uncertainty; three model clusters are too few for a stable model-cluster sandwich estimate. A later, broader study may use two-way clustering when the number of distinct model clusters is adequate. Holm correction covers H1 and H2 only. Every result must report raw rates, denominators, exclusions, effect sizes, and confidence intervals.

The smallest effect of interest is fixed at 10 percentage points, and the analysis formula is set before model data are collected. Comparisons by model or evidence method, unusual curve shapes, rationales, and open-action traces remain exploratory unless a separate study has enough power for them.

7 Evaluation Sizing and Sensitivity

The power analysis does not use language-model outcomes. It uses 10,000 Monte Carlo runs for the instruction comparison and a scenario-clustered linear-probability approximation for the factorial terms. It assumes two confirmatory one-sided tests with Holm correction and the documented random-effect and Bernoulli structure. Power for H1 is not simulated: its precision depends on the as-yet unobserved baseline rate and cluster variation.

Table 4: Prospective estimated power. “Instr.” is the instruction contrast; the final column reports premium-by-risk and premium-by-severity interaction estimates. Values are simulation outputs, not observed effects.

Rep.	Calls	Instr.	Premium	Risk / severity int.
1	2,304	0.991	0.484	0.290 / 0.377
2	4,608	1.000	0.769	0.501 / 0.634
3	6,912	1.000	0.909	0.670 / 0.802

With one repetition, estimated power is 0.991 for the assumed instruction effect. Power is approximately 0.484 for premium, 0.290 for the premium-by-risk interaction, and 0.377 for the premium-by-severity interaction under the stated assumptions. The exact generated values appear in Table 4. The 2,304-choice design is therefore useful for the instruction comparison and for estimating benchmark variance, but not for definitive premium or interaction claims. A later confirmatory study should use blinded pilot estimates to set its sample size.

8 Artifact Controls

The arXiv ancillary archive contains the generated scenarios, JSON schema, hash index, configuration snapshots, seed rules, mock environments, fault schedules, evaluation conditions, run-directory checks, analysis code, tests, bibliography audit, and related-work matrix. The script `paper/build_assets.py` builds the paper’s figures and tables from stored JSON and YAML files.

The accompanying confound report records operation count, serialized plan length, distinct tools, extra verification operations, cost, latency, aliases, plan order, and evidence mechanism for every scenario. Alias set, prompt template, evidence method, and dominant-plan display position are balanced. The diagnostic also confirms two residual couplings: the evidence-dominant plan is necessarily longer, and simulated cost and latency are perfectly correlated. Neither effect can be identified separately in this version.

Evaluation commands reject incomplete preregistration and review files. Fixture outputs are labeled non-empirical and cannot enter the confirmatory analysis. Provider responses go into fixed, uniquely named run directories. Versioned artifacts exclude secrets and API keys. These controls reduce the chance of mixing fixtures, simulations, and model observations. They do not replace independent replication or scientific review.

The ancillary archive is self-contained, carries the Apache License 2.0, and includes the exact scenario manifests and validation reports used by this paper. The release build recreates figures and tables from machine-readable records, compiles the manuscript from a clean unpacked source archive, compares extracted text and page count with the local build, and rejects unresolved citations, missing fonts, unsafe filenames, placeholders, and private or generated cache files.

9 Limitations

Scope of the current release. This release provides evidence about benchmark construction and deterministic execution. It does not include a provider-model evaluation. It therefore cannot show whether agents prefer verifiable plans, whether instructions change that preference, or whether verification improves task success.

Construct validation. Mechanical equivalence does not rule out every wording, salience, or perceived-capability bias. A behavioral study should complete the released blinded human review and a prospectively registered manipulation check before interpreting differences between conditions.

External review. The current release has not undergone peer review or the planned blinded two-reviewer assessment of semantic equivalence and evidence dominance. Automated tests cannot replace those judgments. This paper should therefore be read as a benchmark-and-methods release, not as validated evidence about model behavior.

Synthetic incentives. Cost, delay, failure probability, and consequences are simulated. A language model does not bear these costs in the same way as a person or organization. Its choice may reflect prompt associations rather than a stable attitude toward risk.

Joint premium. Cost and delay change together, so this study cannot separate their effects. That would require a design that varies them independently. Plan length, tool familiarity, and the extra operation may also influence the choice despite neutral names and balanced order.

Evidence independence. Evidence independence depends on the task and system design. A read endpoint backed by the same faulty subsystem may add little useful confirmation. The categorical rubric avoids false numerical precision, but it still includes author judgment.

Generalization and drift. Ninety-six synthetic tasks cannot support broad claims about real systems, agent

designs, or model families. Provider changes and non-deterministic behavior may also make later replication harder, even with fixed seeds.

Literature coverage. The related-work audit is a structured rapid review, not a complete systematic review. Several closely related sources are recent preprints. New work may change the novelty boundary.

10 Ethics

All benchmark actions run in deterministic mock environments. The package makes no real payment, sends no real message, and changes no real account or infrastructure. Scenarios use synthetic entities and limited simulated consequences. The main risk is misinterpretation: readers could mistake a polished benchmark for completed behavioral evidence. The abstract, validation section, and conclusion therefore state clearly what was and was not tested.

A future model evaluation would have financial and environmental costs. It should use a declared budget, bounded outputs, and the smallest design that answers the prospectively specified question. Negative and null results should be retained. This benchmark alone is not enough evidence to deploy a system as a reliability safeguard.

11 Conclusion

PROOFPATHBENCH is an implemented benchmark for studying whether tool-using language agents choose plans that make external outcomes easier to verify independently. Before execution, the agent chooses between two matched plans. The benchmark varies the price of evidence, the stated failure risk, and the consequence severity. It keeps the true state separate from the response shown to the agent. The released software passes deterministic validation across 384 task-treatment units, 768 no-fault plan executions, and nine injected failure types. These results show that the benchmark works as specified. They do not answer the behavioral question. The artifact supports transparent, comparable model evaluations while keeping simulated and prospective values separate from observed results.

AI Assistance Disclosure

Generative-AI tools assisted with software development, literature-search support, manuscript drafting and language editing, and figure layout. No AI system is listed as an author. The named human author remains responsible for the research design, source verification, claims, code, and final text.

Author Contributions

The author designed the study, implemented and validated the software and benchmark, conducted the literature audit and prospective analysis, and prepared the manuscript and release artifact.

Data and Code Availability

The complete benchmark, source code, configuration files, deterministic validation records, tests, and analysis scaffolding are included as an arXiv ancillary archive. The software is released under the Apache License 2.0. No proprietary model output, personal data, credentials, or undisclosed empirical results are included.

References

- Laksh Advani. From confident closing to silent failure: Characterizing false success in llm agents. *arXiv preprint arXiv:2606.09863*, 2026. URL <https://arxiv.org/abs/2606.09863>.
- Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. Cordon: Semantic transactions for tool-using llm agents. *arXiv preprint arXiv:2606.17573*, 2026. URL <https://arxiv.org/abs/2606.17573>.
- Yun-Shiuan Chuang, Chaitanya Kulkarni, Alec M. Chiu, Avinash Thangali, Zijie Pan, Shivani Shekhar, Yirou Ge, Yixi Li, Uma Kona, Linsey Pang, and Prakhar Mehrotra. Toward scalable verifiable reward: Proxy state-based evaluation for multi-turn tool-calling LLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 1251–1264, 2026. doi: 10.18653/v1/2026.acl-industry.87. URL <https://aclanthology.org/2026.acl-industry.87/>.
- Liming Dong, Qinghua Lu, and Liming Zhu. Agentops: Enabling observability of llm agents. *arXiv preprint arXiv:2411.05285*, 2024. URL <https://arxiv.org/abs/2411.05285>.
- Danil S. Grigorev, Alexey K. Kovalev, and Aleksandr I. Panov. VerifyLLM: LLM-based pre-execution task plan verification for robots. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2025. URL <https://arxiv.org/abs/2507.05118>.
- Zongrong Li, Shengkun Ye, Feiyou Guo, and Zuoyou Dang. Silentprobe: Measuring silent failure in production apis used as agent tools. *arXiv preprint arXiv:2609.00035*, 2026. URL <https://arxiv.org/abs/2609.00035>.
- Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. Toolgate: Contract-grounded and verified tool execution for llms. *arXiv preprint arXiv:2601.04688*, 2026a. URL <https://arxiv.org/abs/2601.04688>.
- Yu Liu, Wenxiao Zhang, Zhiwei Yang, Zhongyi Zhang, Hanqi Feng, Xinyu Wang, Peng Qiu, Yanbing Liu, Barnabas Poczoz, and Jin B. Hong. CARE: Pre-execution command verification for shell-executing LLM agents. In *2026 IEEE International Symposium on Software Reliability Engineering*, 2026b. URL <https://arxiv.org/abs/2607.21642>.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark

- for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve llm agent reliability under non-atomic failures. *arXiv preprint arXiv:2608.02645*, 2026. URL <https://arxiv.org/abs/2608.02645>.
- Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. Atomix: Timely, transactional tool use for reliable agentic workflows. *arXiv preprint arXiv:2602.14849*, 2026. URL <https://arxiv.org/abs/2602.14849>.
- Subba Reddy Oota, Francisco Herrera, Jordi Cabot Sagra, Marcos López de Prado, and Shadab Khan. Do LLM agents execute the plans they declare? from planning-mode declaration to pattern-specific execution. *arXiv preprint arXiv:2609.38108*, 2026. URL <https://arxiv.org/abs/2609.38108>.
- Kai Pan and Rong Hou. Agent-first tool api: A semantic interface paradigm for enterprise ai agent systems. *arXiv preprint arXiv:2605.10555*, 2026. URL <https://arxiv.org/abs/2605.10555>.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitit, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=GEcwtMk1uA>.
- Jonathan Sneh, Ruomei Yan, Jialin Yu, Philip Torr, Yarin Gal, Sunando Sengupta, Eric Sommerlade, Alasdair Paren, and Adel Bibi. Tooltweak: An attack on tool selection in LLM-based agents. *arXiv preprint arXiv:2510.02554*, 2025. URL <https://arxiv.org/abs/2510.02554>.
- Jimin Sun, So Yeon Min, Yingshan Chang, and Yonatan Bisk. Tools fail: Detecting silent errors in faulty tools. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14272–14289, 2024. doi: 10.18653/v1/2024.emnlp-main.790. URL <https://aclanthology.org/2024.emnlp-main.790/>.
- Yang Tian, Zhengpeng Shi, and Bo Zhao. Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability. *arXiv preprint arXiv:2606.25819*, 2026. URL <https://arxiv.org/abs/2606.25819>.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjana Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Zihao Wang. Callability is not operability: Controlled interface interventions for llm agents. *arXiv preprint arXiv:2608.23628*, 2026. URL <https://arxiv.org/abs/2608.23628>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in llm agents. *arXiv preprint arXiv:2606.05806*, 2026. URL <https://arxiv.org/abs/2606.05806>.

A Failure Semantics

Table 5: Implemented failure classes and their intended evaluator semantics.

Failure	Authoritative transition	Agent-visible behavior
Explicit failure	No intended change	Explicit error
False success	No intended change	Syntactically valid success
Timeout before	No intended change	Timeout or unknown status
Timeout after	Intended change commits	Same timeout or unknown status
Partial update	Subset of fields commits	Success, partial, or timeout by condition
Stale readback	Current state exists	Older version is returned
Delayed visibility	Write commits; read model lags	Old value until the seeded visibility step
Wrong target	Unintended entity changes	Plausible success for the requested call
Duplicated action	Logical effect applies twice	One or ambiguous response

B Outcome Definitions

VPSR: evidence-dominant registrations divided by eligible registered choices.

VTs: authoritative postcondition and all collateral constraints hold at termination.

FCR: the agent asserts completion while VTS is false.

Recovery success: after an activated fault, the goal is satisfied without a prohibited duplicate or collateral effect.

Uncertainty fidelity: the terminal claim distinguishes verified success, verified failure, and unresolved status correctly.

Every future metric report must include its numerator, denominator, exclusions, confidence interval, and scenario/model clustering strategy.

C Artifact Map

The source package is backed by the following repository paths:

- `benchmark/scenarios/`: 96 generated task manifests;
- `benchmark/runtime_validation_report.json`: deterministic runtime evidence;
- `configs/pilot.yaml`: proposed empirical configuration and gates;
- `proofpath/environments/` and `proofpath/failures/`: mock state and faults;
- `proofpath/evaluation/`: outcomes, metrics, and inferential analysis;
- `research/power_analysis.json`: prospective power outputs;
- `research/confound_diagnostics.json`: design-coupling diagnostics;
- `research/human_validation/`: blinded two-reviewer protocol and blank form;
- `research/related_work_matrix.md`: structured comparison; and
- `research/citation_audit.md`: bibliography verification status.

D Release Scope Statement

The release contains fixture smoke tests but no OpenAI or other provider-model evaluation. Fixtures are explicitly marked non-empirical. The manuscript includes no blank, simulated, or prospective quantity as if it were an observed behavioral result.